

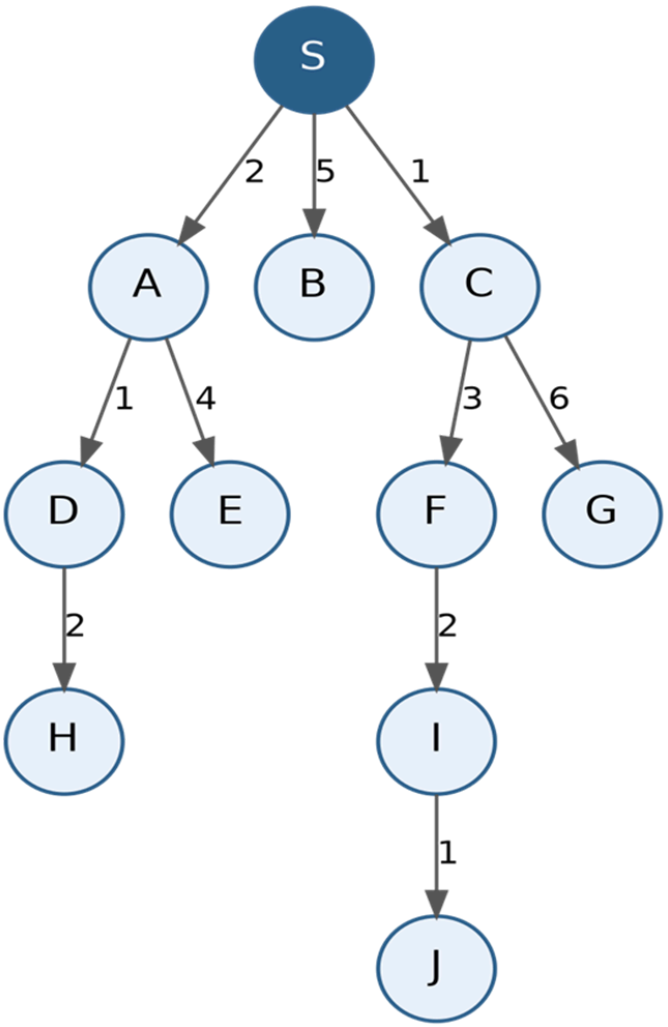
Artificial Intelligence Lab 05
Fall 2026

Uninformed Search Algorithms

Instructor	Ms. Maria Marrium
Lab Instructors	Talha Qureshi Assadullah Farrukh Maha Faisal Hafiz Abdullah Nauman Arif Haris Alam

The Shared Tree

All three tasks below operate on the exact same tree. Edge labels represent traversal cost (used by UCS), whereas for DLS and IDS, treat the tree as unweighted and reason purely in terms of depth/hops.



Adjacency List Representation

```
tree = {
        'S': [('A'), ('B'), ('C')],
        'A': [('D'), ('E')],
        'B': [],
        'C': [('F'), ('G')],
        'D': [('H')],
        'E': [],
        'F': [('I')],
        'G': [],
        'H': [],
        'I': [('J')],
        'J': []
}
```

Task 1: Depth-Limited Search on the Tree

Scenario

A network engineer needs the route from the root server 'S' to a target node. The network has a policy: no message may be relayed through more than a fixed number of intermediate nodes, regardless of cost. You must check if the target is reachable within that hop restriction.

Instructions

- Implement def dls_search(tree, start, goal, depth_limit):
- Cut off and do not expand any branch once depth_limit is exceeded.
- Return a valid path (list) if the goal is found within the limit, else None.
- start == goal should return [start] at depth 0.

Sample Input / Output

➔ dls_search(tree, 'S', 'J', 3):
None

➔ dls_search(tree, 'S', 'J', 4):
['S', 'C', 'F', 'I', 'J']

Task 2: Iterative Deepening Search on the Tree

Scenario

The engineer no longer knows a safe hop limit in advance. Instead of guessing, they want to try depth 0, then 1, then 2... until the target is found, combining DLS's memory efficiency with BFS's guarantee of finding the shallowest solution.

Instructions

- Implement def ids_search(tree, start, goal, max_depth):
- Internally call a depth-limited routine repeatedly, increasing the limit from 0 up to max_depth.
- Print each depth limit attempted, and whether it succeeded or failed at that depth.
- Return (path, depth_found) for the first depth at which the goal is found, or None if not found by max_depth.

Sample Input / Output

```
➔ ids_search(tree, 'S', 'J', 5)

   Trying depth limit 0... Failed
   Trying depth limit 1... Failed
   Trying depth limit 2... Failed
   Trying depth limit 3... Failed
    Trying depth limit 4... Success
   Path: ['S', 'C', 'F', 'I', 'J']
   Depth Found: 4
```

Task 3: The Vault-breaker's Dilemma

Scenario

A safecracker is breaking into a vault built as nested floors, each floor connected downward through several possible hatches, each hatch triggering a different amount of guard suspicion (a cost) if used. The prize could be sitting on any floor while the safecracker has no idea how deep but going in blind and diving straight to the bottom is far too risky, so they insist on checking floor by floor, shallowest first, never considering a deeper floor until every hatch on the current floor has been ruled out.

Once the prize is finally spotted on some floor, the safecracker doesn't just want a way in. If there are multiple hatch-routes that reach the prize at that same shallowest floor, they want the one that raises the least suspicion overall.

The Vault (adjacency list)

```
vault = {
        'Entry': [('A'), ('B'), ('C'), ('G'), ('J'), ('K'), ('N')],
        'A': [('D')],
        'B': [('D'), ('E')],
        'C': [('Vault'), ('Vault')],
        'D': [('Vault')],
        'E': [('Vault')],
        'G': [('H')],
        'H': [('I')],
        'I': [('Vault')],
        'J': [('Vault')],
        'K': [('L')],
        'L': [('M')],
        'M': [('Vault')],
        'N': [('O')],
        'O': [('P')],
        'P': [('Q')],
        'Q': [('Vault')],
        'Vault': []
}
```

Task

```
def crack_vault(vault, entry, prize):
        ...
```

Fine Print

- The safecracker must be certain no shallower floor holds the prize before considering a deeper one.
- Among all routes that reach the prize on that shallowest possible floor, only the cheapest (least suspicion) one should be reported.
- entry == prize should return immediately with cost 0.
- Unreachable prize should return None.

Hint-free zone: Think carefully about what “shallowest first” demands !!

Sample Input / Output

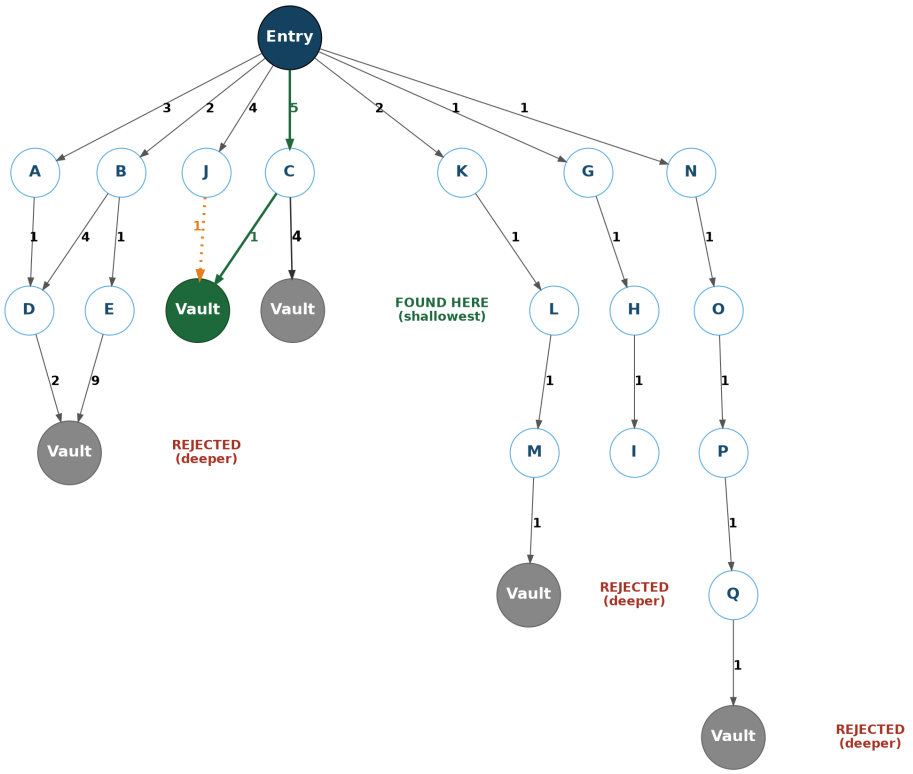
```
➔ crack_vault(vault, 'Entry', 'Vault')

  (['Entry', 'C', 'Vault'], 2, 1)
```

Note: As you can see in the example, the shallowest Vault node is present at floor 2 depth 1 while there are 2 Vault nodes present (Vault 1 is reachable via C and J both with cost 1 , while the other vault node is reachable via C with cost 4). If such a scenario exists then you'll have to choose the vault node such that that specific node -> vault's path is the cheapest one among all the node -> vault paths on that floor. In the above condition, the second vault node that is reachable via C with cost 4 from C -> Vault is rejected because a cheaper path from direct node to the vault already exists i.e. to first Vault node either through J or C. Now, as the cost of reaching the vault through both the nodes is equal i.e. 1, in this case you may return the one that would alphabetically appear first. In the given scenario, the expected answer would be the C node.

Visual Walkthrough (using the example above)

The diagram below traces the exact same vault from the sample input: Floor 0 is Entry, Floor 1 is every room directly reachable from Entry, and Floor 2 is where Vault is first spotted via both C and J.



Cost only ever gets compared within the same level. So if two routes both reach Vault at Level 2, say one node to vault costing 6, another costing 5, then (and only then) does the cheaper one (5) win. Cost is a tie-breaker for equal depth, never a way to justify going deeper.